

HATA DEFTERİ

Autonomous car — defect log of a previous build

MEB Uluslararası Robot Yarışması · Otonom Araç

Snapshot built 02 August 2026

This PDF is a read-only snapshot. The living document is `HATA_DEFTERI.md` ; `PLAN_New.md` is authoritative above both.

Read section 0 before the defect list. A fault list read on its own gives a false impression of the build it describes, and section 0 is not padding.

The output of this document is section 7 — seven guards. The list of faults is only the evidence for them.

Contents

HATA DEFTERİ — the LEGACY defect log

0. Read this before the list
 - What the May car actually did well
 - The actual failure mode
1. Summary
2. Driving faults
 - 1 — The perspective quad was never rescaled
 - 2 — The motor trims were never measured
 - 3 — The balance tool writes names the config does not read
 - 8 — No PWM frequency is set anywhere
3. Points left on the table
 - 6 & 7 — sign_type is read but never written
 - 11 — The physical start button was removed
4. Safety
 - 9 — Errors are caught, motors keep running
5. Why none of it was noticed
 - 10 — Telemetry stopped halfway through every race
 - 14 — Four fabricated documents
6. Latent, hardware and drift
 - 4 & 5 — Two trim bugs, currently masked
 - 12 — Detectors disagree about brightness
 - 13 — Two config files
 - 15 & 16 — Hardware margins
 - 17 — Documentation drift, the honest kind
7. The guards, collected
8. Closing note

HATA DEFTERİ — the LEGACY defect log

Every fault found in the 4 May 2026 build, with evidence, cause, cost and the guard that prevents it recurring.

This is a derived document. `PLAN_New.md` is authoritative; every entry cites the plan section it came from. If the two disagree, the plan is right.

0. Read this before the list

A defect list read on its own gives a false impression, so this part is not padding.

What the May car actually did well

- **Bird's-eye perspective warp** before lane finding, rather than chasing pixels in a camera-angle image.
- **CLAHE on the L channel** to normalise spot reflections before thresholding.
- **Adaptive white profiles** — DARK / NORMAL / BRIGHT chosen per frame from mean brightness, instead of one fixed threshold.
- **Column-continuity weighting.** A lane line is vertically continuous in bird's-eye view; a glare spot is not. Weighting histogram columns by their vertical coverage filters reflections almost for free. This is a genuinely clever idea and it is original.
- **Near/far weighted error** — a lookahead term for curves plus a near term for position.
- **Lane memory with a narrowed search window**, which is also what already handles dashed centre lines.
- **PD control with dynamic gain, derivative capping, curve-speed coordination and dead-zone compensation.**
- **Event debouncing** — a detection must persist N frames before it fires.
- `gpiozero` and `picamera2`, both import-guarded for off-Pi development. Most people get the Pi 5 GPIO story wrong; this build did not.
- **A HOG sign classifier** with rotation and brightness augmentation.
- **Nine calibration tools**, including a live tuner that writes back to config.

That is a strong build. Hold onto that while reading the rest.

The actual failure mode

Read the **Cause** column in every entry below. Almost none of these are knowledge failures — nobody here failed to understand adaptive thresholding or PID control. They are **seam failures**, and nearly every seam crosses a session boundary or a tool boundary:

- The camera resolution changed in one place; the perspective quad lived in another.
- Trim grew from two variables to four; the tool that writes them stayed on two.
- A sign model was trained by one effort; its consumer was written by another; the wire between them was never run, though both sides spell the labels identically.
- Documentation was generated describing intended work, then trusted as a record of finished work.

And the instrumentation was lying. Four documents dated eleven days before the competition asserted  TÜM OPTİMİZASYONLAR UYGULANMIŞTIR, ± 15 px deviation, 28–30 FPS, +85–95 puan. None of it was measured. Being wrong while your own reports say you are fine is not stupidity — it is the predictable result of a tool that confabulates and a process with no verification step.

The guards in this document exist so the seams cannot silently open again.

1. Summary

#	Defect	Category	Cost
1	Perspective quad never rescaled after resolution change	Driving	Suspected primary cause of the road-following failure
2	Motor trims never measured	Driving	Constant pull to one side
3	Balance tool writes variable names nothing reads	Tooling	Made #2 undetectable even if run
4	Trim applied twice	Latent	Squares the correction
5	Trim selected by sign, not by wheel	Latent	Correction cannot correct anything
6	<code>sign_type</code> consumed but never produced	Points	Çıkamaz yol, 100 points , unreachable
7	Trained sign model never connected	Points	Same root as #6
8	No PWM frequency set anywhere	Driving	100 Hz default; probable cause of the dead-zone hack
9	Error handler does not stop motors	Safety	Up to ~1.2 s driving blind
10	Log stops at 120 s of a 240 s race	Diagnosis	Half of every run unrecorded
11	Physical start button removed	Points	50 points + rule exposure
12	Crosswalk uses fixed threshold while lanes adapt	Robustness	Detectors disagree under venue light
13	Two config files, one corrupted	Drift	Divergent values
14	Four fabricated documents	Process	The reason none of the above was caught
15	Motors driven at ~175 % of rated voltage	Hardware	Amplifies every control error
16	Two motors share one 2 A channel	Hardware	Both stall together on the bump
17	<code>CLAUDE.md</code> state list drifted from code	Drift	Phantom state, three real ones missing

2. Driving faults

1 — The perspective quad was never rescaled

Evidence. LEGACY/config.py:23–24

```
# ⚠️ 800x680 çözünürlük için yeniden kalibre edilmeli (calibrate.py çalıştırın).  
PERSP_SRC = [[160, 300], [480, 300], [0, 480], [640, 480]]
```

WIDTH = 800, HEIGHT = 680. Those coordinates describe a **640x480** frame.

Cause. The capture resolution was raised and every *other* resolution-dependent constant was migrated — ROAD_ROI_BOTTOM = 680 proves the pass happened. This one was flagged with a warning and skipped, and the warning was never actioned.

Cost. The quad's horizontal centre is x=320; the frame's is x=400. lane.py measures error against bird_w // 2 as though the warp were centred. It is not. A perfectly centred car reads roughly **–100 px of error**, so at KP = 0.30 the controller applies about 30 PWM of permanent correction to a car that is exactly where it should be. **No amount of PD tuning fixes this** — the controller is faithfully correcting an error that does not exist. Separately, the quad's bottom edge sits at y=480 in a 680-row frame, so the nearest 200 rows of road never enter the warp at all.

For scale: ASSUMED_LANE_WIDTH = 300. The bias is about a third of a lane.

Guard. Resolution-dependent constants are derived from WIDTH / HEIGHT, or validated against them at startup and **refused** if inconsistent. A warning comment is not a guard; a failing check is. → Plan §3.1, §20.8

2 — The motor trims were never measured

Evidence. LEGACY/config.py:101–104 — all four at 1.0.

Cause. Step 1 of the team's own tuning workflow (BASLA_BURADAN.txt). Never completed, or completed and lost. See #3 for why it may have looked completed.

Cost. Cheap gearmotors are never matched. On a differential-drive car an uncorrected imbalance is a permanent pull, and the PD controller spends the run fighting a bias instead of following the road. With KI = 0.04 the integral partly masks it on straights, then delivers the accumulated bias into the next corner.

Guard. kontrol.py fails if any trim is still exactly 1.0. → Plan §3.2

3 — The balance tool writes names the config does not read

Evidence. LEGACY/motor_balance_test.py:83–85 prints:

```
LEFT_TRIM = 0.95  
RIGHT_TRIM = 1.0
```

config.py reads LEFT_TRIM_LOW, LEFT_TRIM_HIGH, RIGHT_TRIM_LOW, RIGHT_TRIM_HIGH.

Cause. DOSYALAR_GUNCELEME_DURUSU.txt records it exactly: config.py was given speed-dependent trim profiles, while motor_balance_test.py was explicitly left ORIGINAL. Two files changed independently and were never reconciled.

Cost. This is the cruel one. The test could have been run correctly, its output pasted in exactly as instructed, and **nothing would have happened** — a dead variable sitting in config.py while the four live ones stayed at 1.0. The work was possibly done and was structurally incapable of having an effect.

Guard. A tool that writes configuration **imports the names it writes**, so a rename breaks it loudly instead of silently. → Plan §20.3e

8 — No PWM frequency is set anywhere

Evidence. LEGACY/motor.py:46-47

```
self._right_pwm = PWMOutputDevice(RIGHT_PWM_PIN)
self._left_pwm  = PWMOutputDevice(LEFT_PWM_PIN)
```

No frequency argument, and no `PWM_FREQ` anywhere in `config.py`. `gpiozero` defaults to **100 Hz**. The earlier CascadeProjects build set `PWM_FREQ = 1000`; the setting was lost in a rewrite.

Cost. 100 Hz is coarse for motor control and probably explains `DEAD_ZONE_MIN_PWM = 30` — a minimum-power floor is exactly the workaround you reach for when low duty cycles produce no usable torque. The hack may be treating a symptom of the frequency.

Guard. Set it explicitly with a comment recording the measurement it came from. Test whether the dead zone shrinks before inheriting the hack. → *Plan §13*

3. Points left on the table

6 & 7 — `sign_type` is read but never written

Evidence. LEGACY/main.py:356

```
sign = events.get('sign_type')
if sign == 'cikmazsokak': → ÇIKMAZSOKAK, brake, turn right
elif sign == 'sollamabam': → open the no-overtaking window
```

events.py:244–255 returns exactly `traffic_light`, `crosswalk`, `crosswalk_close`, `hemzemin`, `hemzemin_close`, `speed_bump`, `orange_car`, `yellow_car`, `parking_zone`, `sign_blue`. **There is no `sign_type` key.** `events.get('sign_type')` is always `None` and both branches are unreachable.

Meanwhile `sign_model.json` holds a trained classifier whose classes are:

```
['cikmazsokak', 'hemzemin', 'kasis', 'park', 'sollamabam', 'yayagecidi']
```

Both strings `main.py` tests for are present, spelled identically. The model was trained *for* that consumer. It is referenced only by `train_sign.py` and `sign_test.py` — never by the running system.

Cost. ÇıkmaZ yol is worth 100 points and could never be scored. The handling state is written correctly and is never entered. The no-overtaking window never opens either, so overtaking was permitted everywhere on the track; only the `not events['yellow_car']` check remained, which is a fallback rather than the rule.

Cause. A model built by one effort, a consumer written by another, and no step that checked the two ends met.

Guard. The event dictionary gets a **single declared schema**. A consumer reading a key no producer writes fails loudly at startup rather than returning `None` for a season. → Plan §3.5, §20.3c

11 — The physical start button was removed

Evidence. LEGACY/main.py:9 — `GPIO 16 buton kaldırıldı`. Documented start paths are typing `GG` or `EZ`, or pressing `SPACE`, read through `termios/tty`.

Cost. The guide awards **50 points** for starting from a physical button with no external computer. A keyboard start forfeits them and requires a terminal attached to the car, which sits close to the no-external-computer rule.

Note. This is the same `GG / EZ` hotkey that `CLAUDE.md` names as its example of a change to reject for having no reason behind it. → Plan §3.4

4. Safety

9 — Errors are caught, motors keep running

Evidence. LEGACY/main.py:517-525

```
except Exception as _e:
    _err_count += 1
    print(f"[main] KARE HATASI ({_err_count}): {_e}")
    if _err_count > 30:
        _running = False
```

The loop prints and continues. **It never brakes.** Whatever PWM was last commanded stays applied for up to 30 consecutive failures — at 25 FPS, over a second of a powered car driving on stale commands.

Guard. The new loop **brakes first, then logs.** This is already the standing rule in CLAUDE.md : motors default off on startup and on any error, never assuming a previous safe state. The legacy code does not honour it. → *Plan §20.8*

5. Why none of it was noticed

10 — Telemetry stopped halfway through every race

Evidence. `LEGACY/config.py:140` — `LOG_DURATION_SEC = 120`. The race limit is **240 seconds**.

Cost. The only record the car kept switched itself off at the halfway point. A failure in the back half of the course was never going to leave evidence. Combined with rule 2.3 (radio off, so no SSH, no screen, no dashboard), the car came back with nothing to say.

Guard. `kayit.py` runs for the full race plus margin, and `kontrol.py` fails if the log duration is less than the race duration. → *Plan §21.6*

14 — Four fabricated documents

Evidence. `BASLA_BURADAN.txt`, `DOSYALAR_GUNCELEME_DURUSU.txt`, `DEGISIKLIKLER_OZET.txt`, `IMPLEMENTATION_SUMMARY.txt`, all dated 2026-04-25.

Nine constants described in detail that exist in no file — including `SHARPNESS_THRESHOLD_HIGH` and `SHARPNESS_THRESHOLD_MED`, for a sharpness feature that exists nowhere in the project in any version.

Five methods described with their behaviour that do not exist: `_get_adaptive_hsv()`, `_get_adaptive_kp()`, `_get_adaptive_kd()`, `_apply_response_mapping()`, `_apply_dead_zone_compensation()`. The last is informative — the real method is `_apply_dead_zone_pair()`, so the docs describe a **renamed predecessor**.

Ten filenames referenced that do not exist, including `FINAL_DELIVERY.md`, which `BASLA_BURADAN.txt` names as the single most important file in the project.

Wrong values: `KP` claimed `0.60`, actually `0.30`. Derivative cap claimed `±600`, actually `150`. Line counts claimed `252/205/153`, actually `293/253/184`.

Invented measurements: `±40 px → ±15 px`, `24–26 → 28–30 FPS` (adding per-frame CLAHE cannot *raise* frame rate), `+85–95 puan` itemised per feature.

`BASLA_BURADAN.txt` signs off with `DOSYA KONUMU: /mnt/user-data/outputs/` — an AI sandbox path, copied verbatim into the repository.

Why it survived. The reports are not fiction. Six of their headline features genuinely exist — adaptive HSV, CLAHE, dead-zone compensation, speed-dependent trim, dynamic gain, derivative capping are all really in the code. **The architecture claims are true and every specific number, name, file and metric is invented.** Check two or three claims, find them correct, stop checking. A document that was wholly wrong would have been discarded in a minute.

Guard. The rule now in `CLAUDE.md`: documentation may describe only what has been read in the code or measured on a run; if a document names a constant, method or filename, that name must be findable with `grep`. Every fabrication above fails that test **mechanically, in about ten seconds**. → *Plan §21*

6. Latent, hardware and drift

4 & 5 — Two trim bugs, currently masked

Invisible while the trims are `1.0`, because multiplying by one twice changes nothing. They activate the moment real values are measured — i.e. the moment someone fixes #2.

Wrong wheel. `controller.py:165`

```
trim = LEFT_TRIM_LOW if pwm >= 0 else RIGHT_TRIM_LOW
```

Selected by the **sign of the value**, not by which wheel it belongs to. Called once for left and once for right, so whenever both drive forward **both receive the LEFT trim**. A correction applied identically to both wheels cannot correct an imbalance between them.

Applied twice. `controller.py:109–110` trims, then `motor.py:65–69` trims the same value again, that time correctly per wheel. Net: `LEFT_TRIM2` on the left, `LEFT_TRIM × RIGHT_TRIM` on the right.

`LEGACY/CLAUDE.md` records the double application as "*intentional in current code*" — a bug laundered into a design decision, which is how it survived review.

Guard. Trim lives in exactly one layer (the motor driver) and is selected by wheel identity, never by the sign of a number. → *Plan §3.3*

12 — Detectors disagree about brightness

`events.py` calls the fixed `WHITE_HSV_LOW/HIGH` for crosswalk stripes, while `lane.py` picks a DARK/NORMAL/BRIGHT profile from the scene. Under venue lighting the two can disagree about what counts as white.

Guard. One brightness decision per frame, shared by every detector. → *Plan §20.8*

13 — Two config files

`config.py` and `config_to_be_migrated.py`. The second carries the same stale `Persp_src`, the same `1.0` trims, older gain multipliers (`1.0` where the live file has `1.3 / 1.2`), and its Turkish comments are **mojibake** (`TÃ¼rk` — UTF-8 read as Latin-1). A corrupted older fork with nothing unique in it. → *Plan §20.3b*

15 & 16 — Hardware margins

Overvoltage. The L298N runs from 3×18650 , so 6 V motors see roughly 9–10.6 V — about **175 % of rating** — while `config.py` sets `BASE_SPEED = 62`, `MAX_SPEED = 85`. An overvolted motor is faster and more torquey than the gains were tuned for, so every correction lands harder than intended and tuning becomes much harder.

Current. Four motors, two paralleled per L298N channel, sharing one channel's ~2 A budget. Two gearmotors stalling together — exactly what a speed bump causes — will exceed it, and the overvoltage raises stall current further.

Guard. Measure at the terminals under load before trusting either. `max_pwm` starts near 57 %, not 85 %. → *Plan §3.6, §3.0*

17 — Documentation drift, the honest kind

`LEGACY/CLAUDE.md` is **broadly accurate** — its descriptions of the lane pipeline, debouncing, the logger and the gpiozero wrapper all check out. Its errors are drift, not invention: it lists a state `KIRMIZI_ISIK` that exists in no file, and omits three that do (`YAYA_YAKLAS`, `HEMZEMIN_YAKLAS`, `CIKMAZSOKAK`), so the best design in the state machine — the two-stage approach — is undocumented.

This contrast is the lesson. `CLAUDE.md` was written by *reading the code* and is reliable. The four reports in #14 were written by an assistant *describing work it believed it had done*, and are reliable about nothing. Same tool, same project, opposite trustworthiness — and the difference is only whether the author looked at the artifact. → *Plan §21.3, §21.5*

7. The guards, collected

Everything above reduces to seven rules. They are the actual output of this document.

1. **Resolution- and hardware-dependent constants are derived or validated at startup.** A warning comment is not a guard; a failing check is.
2. **A tool that writes configuration imports the names it writes.**
3. **Shared data structures have a declared schema.** Reading an undeclared key fails loudly.
4. **Motors brake first, then the error is logged.**
5. **Trim lives in one layer and is selected by wheel identity.**
6. **Telemetry runs for the whole race**, and a check refuses to pass if it does not.
7. **Documentation describes only what was read or measured.** Any named constant, method or file must be findable with grep, and any performance number must carry the date of the run it came from.

Rules 1, 2, 3 and 6 become assertions in `kontro1.py`, so they are enforced by a script rather than by memory. Rule 7 is in `CLAUDE.md`. Rules 4 and 5 are design constraints on the rewrite.

8. Closing note

Seventeen defects sounds damning until you notice that thirteen of them are seams between correctly-built parts, and that the four documents which should have caught them were instead asserting that everything had been optimised and measured.

The car was closer to working than it looked. That is worth knowing, and it is the reason §20.7 says to fix the quad, measure the trims and drive it **before** deciding how much to rewrite.